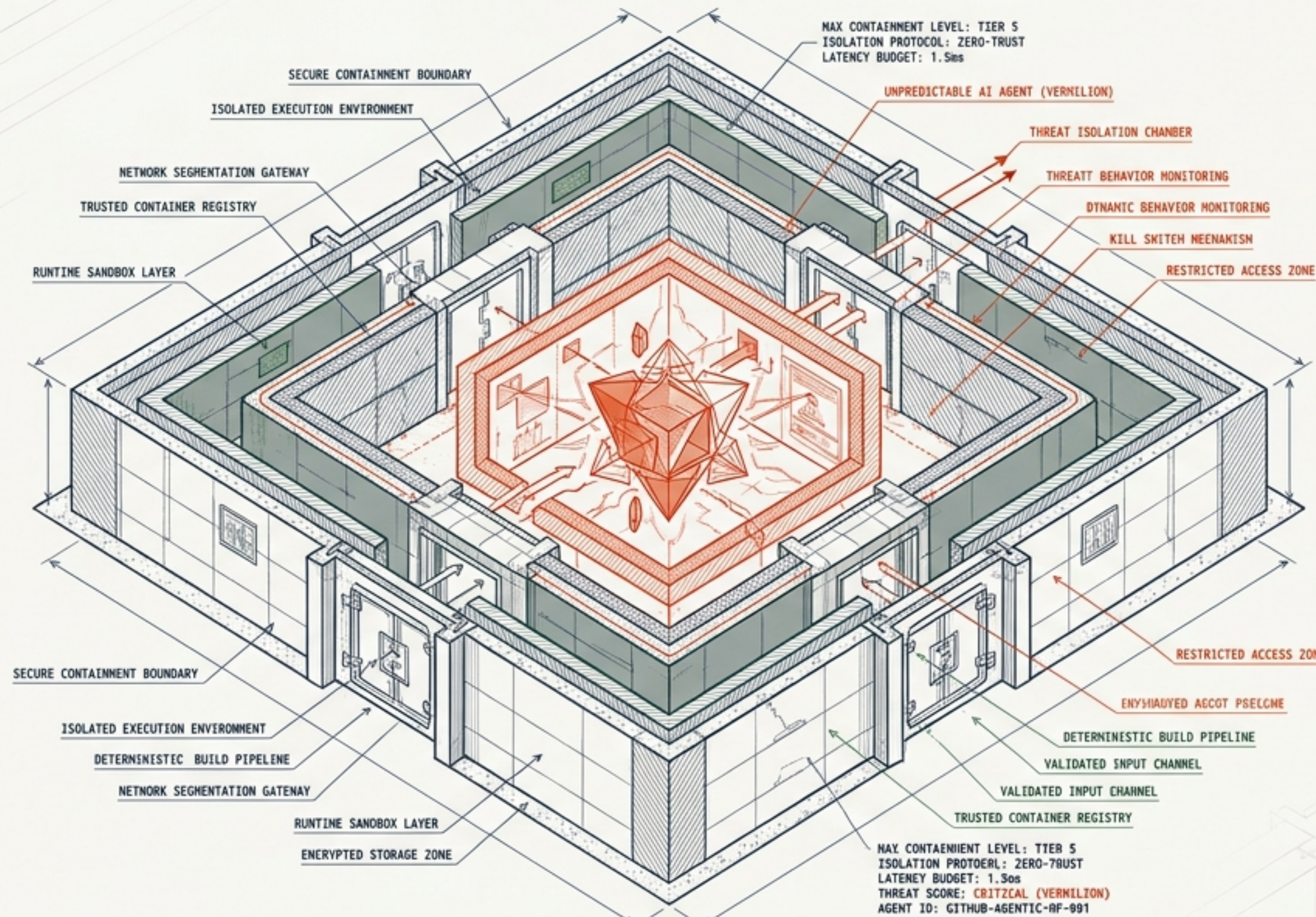




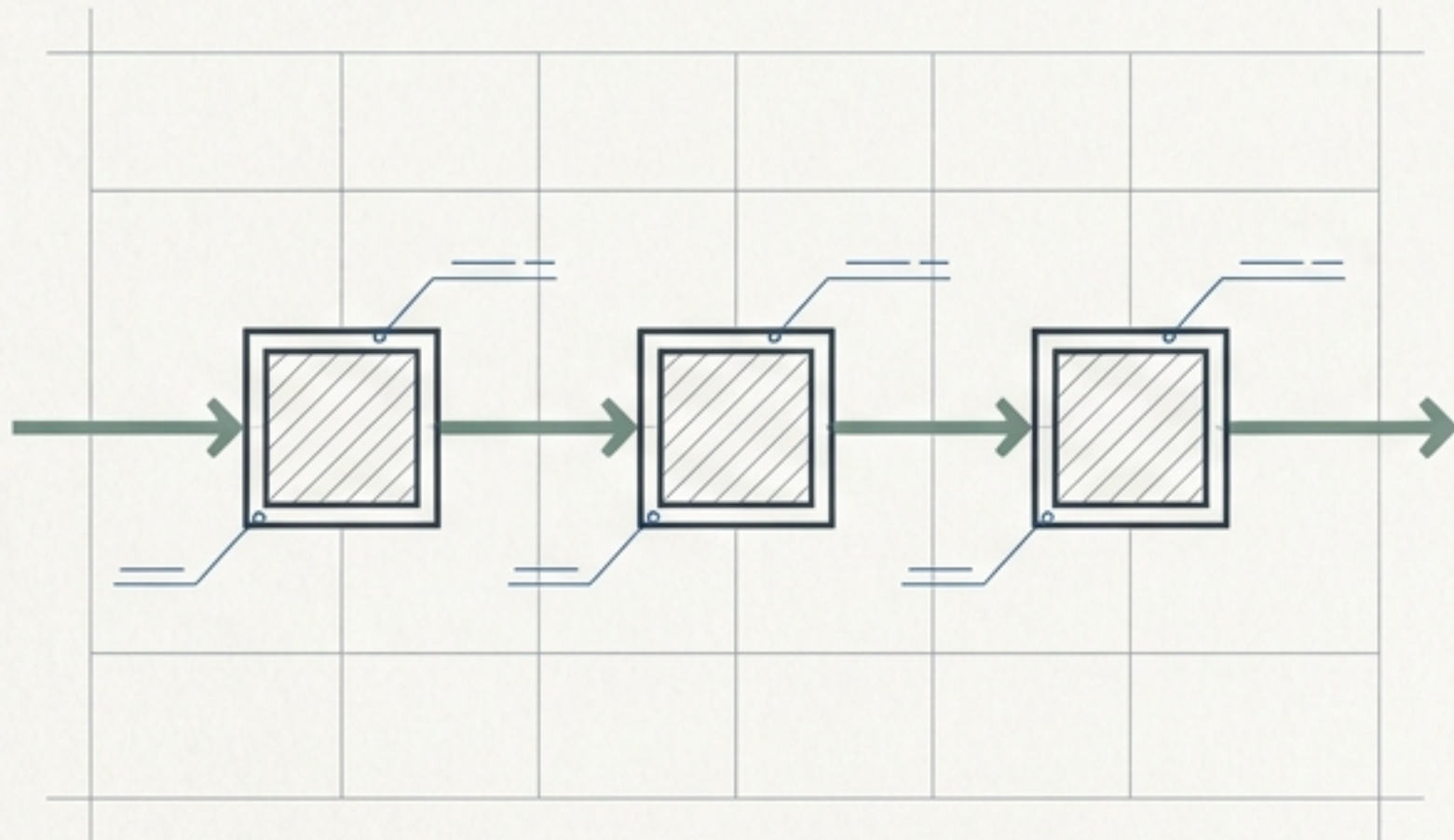
การกักกัน AI Agent ใน CI/CD

สถาปัตยกรรม Zero-Trust สำหรับระบบที่ไม่สามารถคาดเดาได้ (ถอดรหัสจาก GitHub Agentic Workflow)

AI Agent ทำลายกฎพื้นฐานของ CI/CD

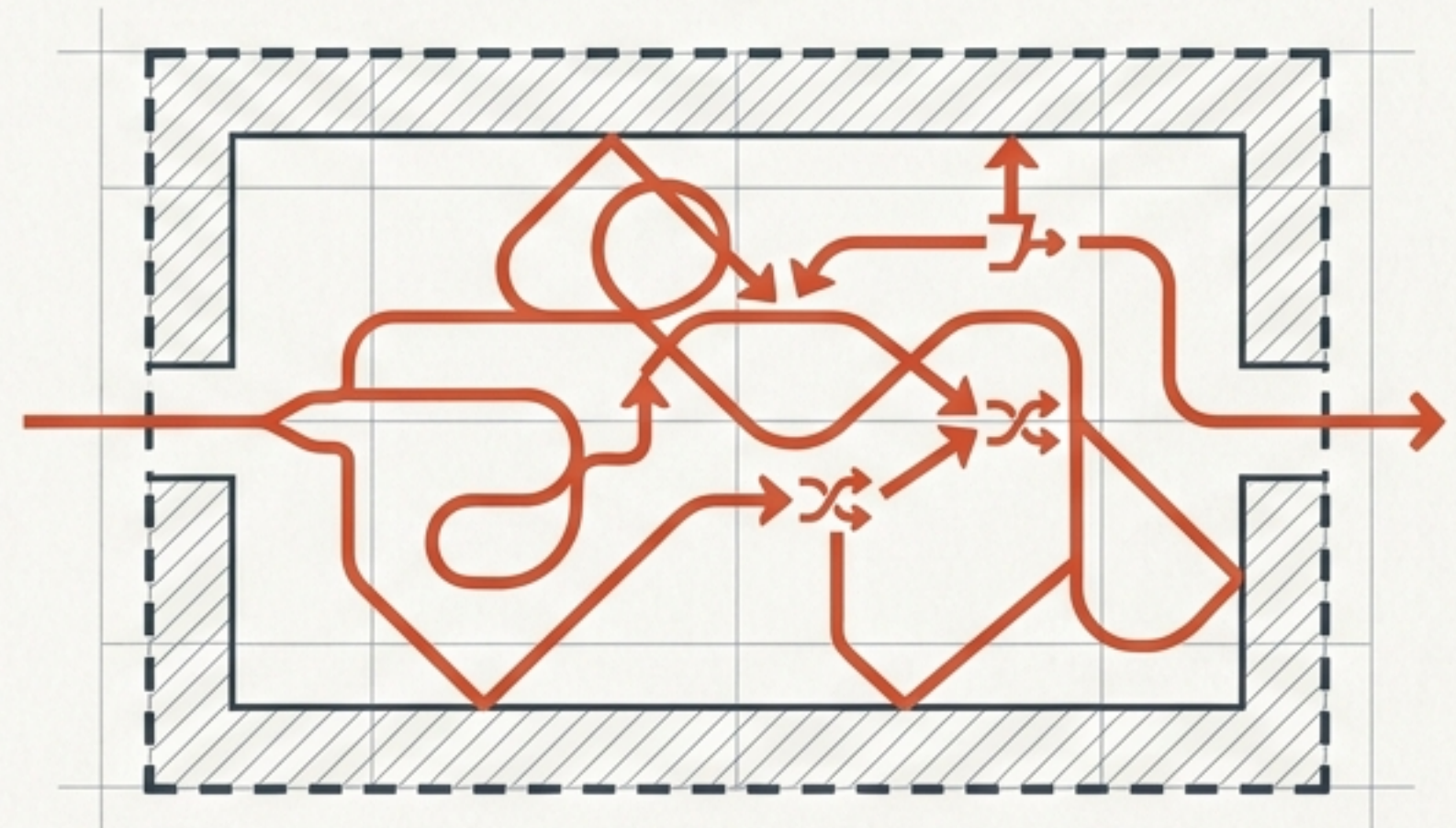
ไปป์ไลน์ CI/CD แบบดั้งเดิมถูกสร้างขึ้นบนพื้นฐานความไว้วางใจ (Shared Trust Domain) เพราะสคริปต์ทุกตัวมีพฤติกรรมที่คาดเดาได้ แต่ Agent ไม่ได้ทำตามสคริปต์ตายตัว มันอ่านบริบทและตัดสินใจเอง ซึ่งแปลว่ามันสามารถถูกหลอกได้

Traditional Automation



Deterministic (ทำตามคำสั่งแบบ 100%)

AI Agents



Non-Deterministic (ตัดสินใจเองแบบ Real-time)

Paradigm Shift: จากสคริปต์สู่ตัวแทนที่คิดเองได้

มิติการเปรียบเทียบ	Traditional CI/CD	Agentic CI/CD
รูปแบบการทำงาน (Execution)	Deterministic (คาดเดาได้)	Runtime Decisions (ประมวลผลและตัดสินใจหน้างาน)
โมเดลความปลอดภัย (Trust)	Shared Domain (แชร์ความลับร่วมกัน)	Zero Trust (แยกส่วนประกอบอย่างเด็ดขาด)
ความเสี่ยงหลัก (Failure Mode)	Expected Crash (โศกพั้ง/บ๊ึก)	Malicious Action (เช่น Prompt Injection)

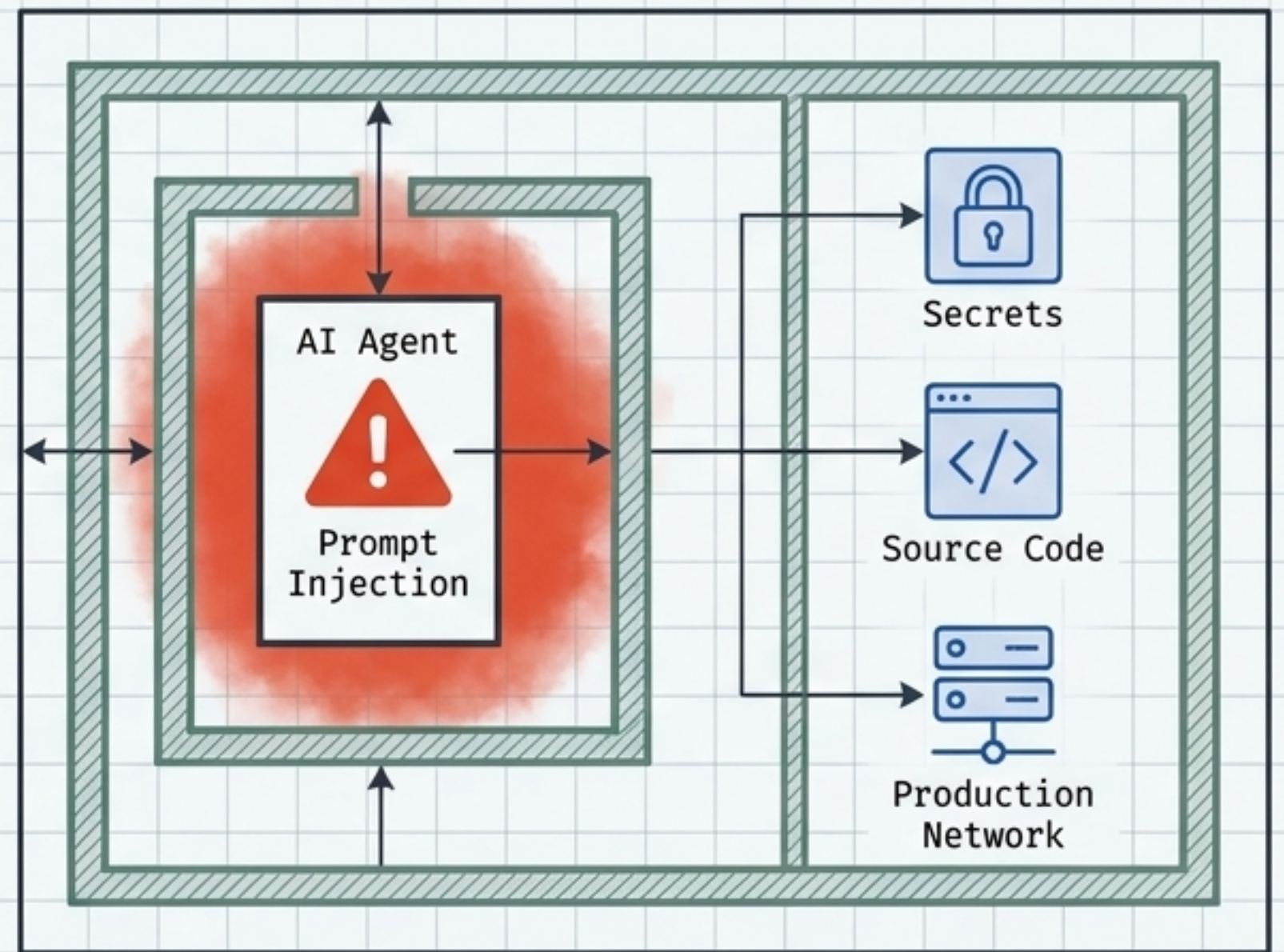
สมมติฐานหลัก: 'Agent ถูกเจาะระบบเรียบร้อยแล้ว'

การนำ Agent เข้าสู่ CI/CD ต้องออกแบบภายใต้สมมติฐานที่เลวร้ายที่สุด หาก Agent โดน Prompt Injection มันจะพยายามขโมย API Keys, สแปม Repository และเจาะระบบเครือข่าย เราจึงต้องจำกัดขอบเขตความเสียหาย (Blast Radius) ให้แคบที่สุด

Shared Runner (Traditional)



GitHub's Architecture (Zero-Trust)



ปรัชญาการออกแบบ: Strict-by-Default

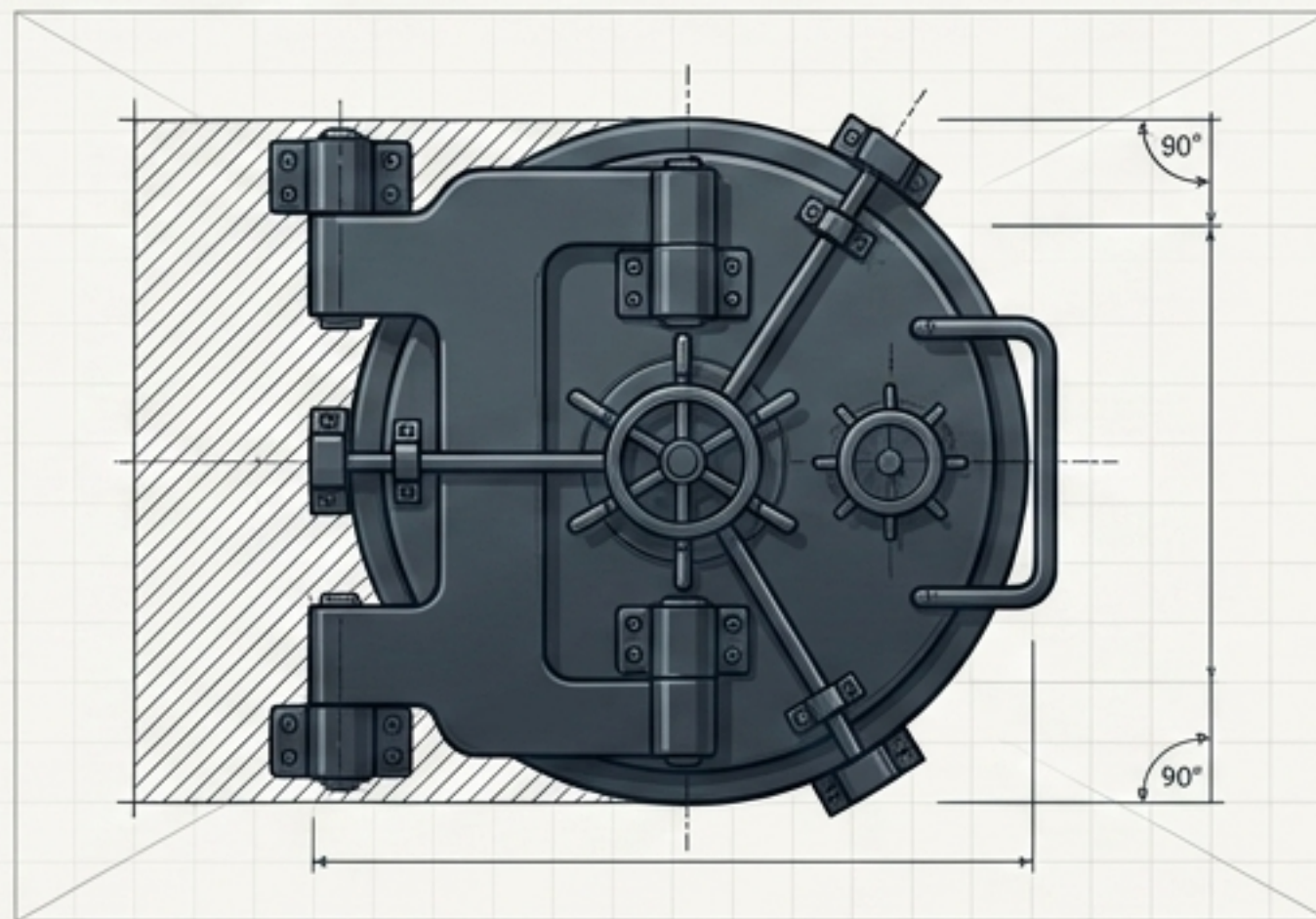
เครื่องมือ AI ส่วนใหญ่มักให้ผู้ใช้เลือกเปิดระบบความปลอดภัยเอง (Opt-in) เพื่อความสะดวก แต่ GitHub เลือกใช้โหมด Strict-by-Default ยอมแลกความยืดหยุ่นของนักพัฒนาเพื่อให้มั่นใจว่าความปลอดภัยคือค่าเริ่มต้นที่ไม่สามารถข้ามได้

Opt-in Security (Consumer Tools)



Security is Optional

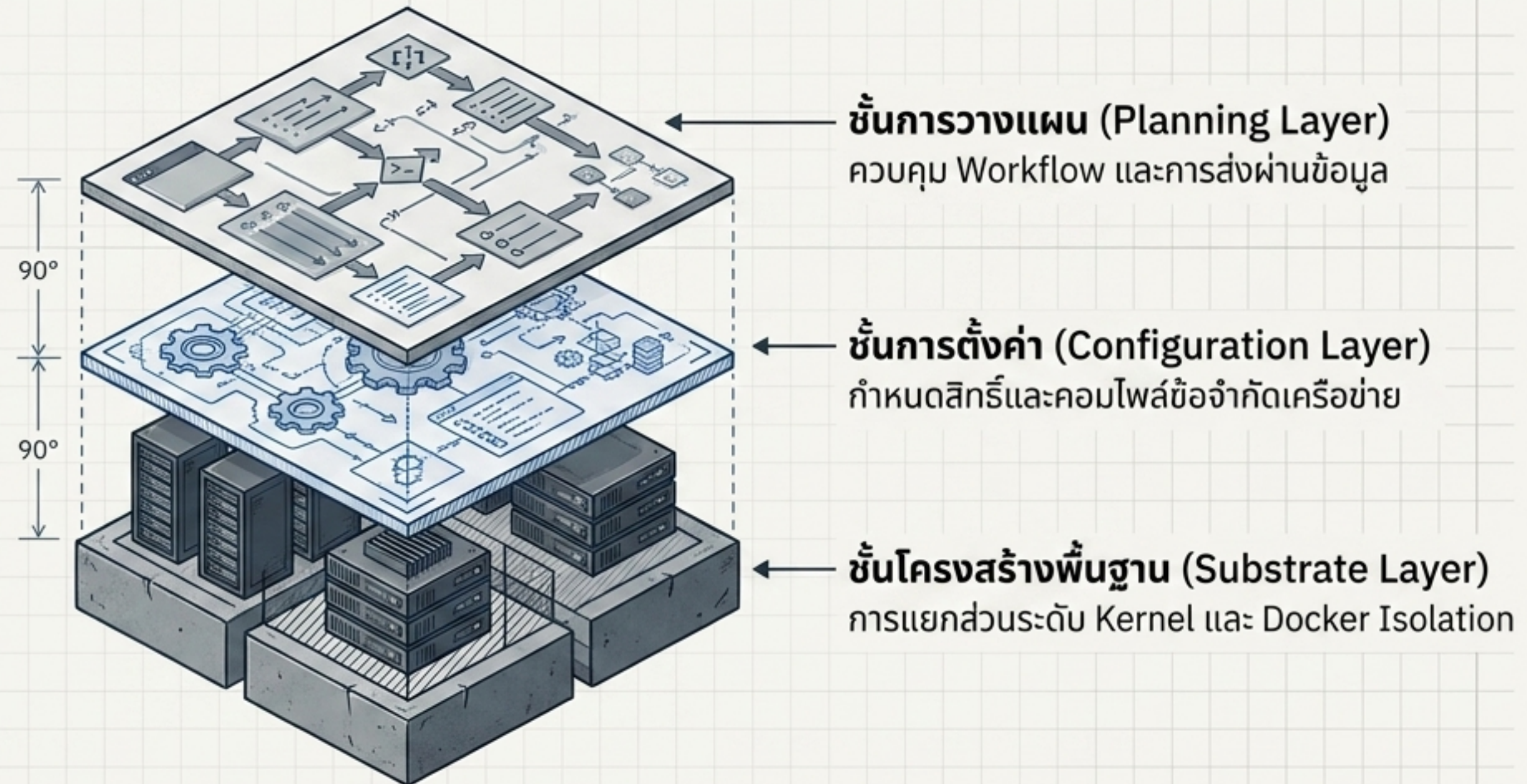
Strict-by-Default (Enterprise Workflow)



Security is Mandatory

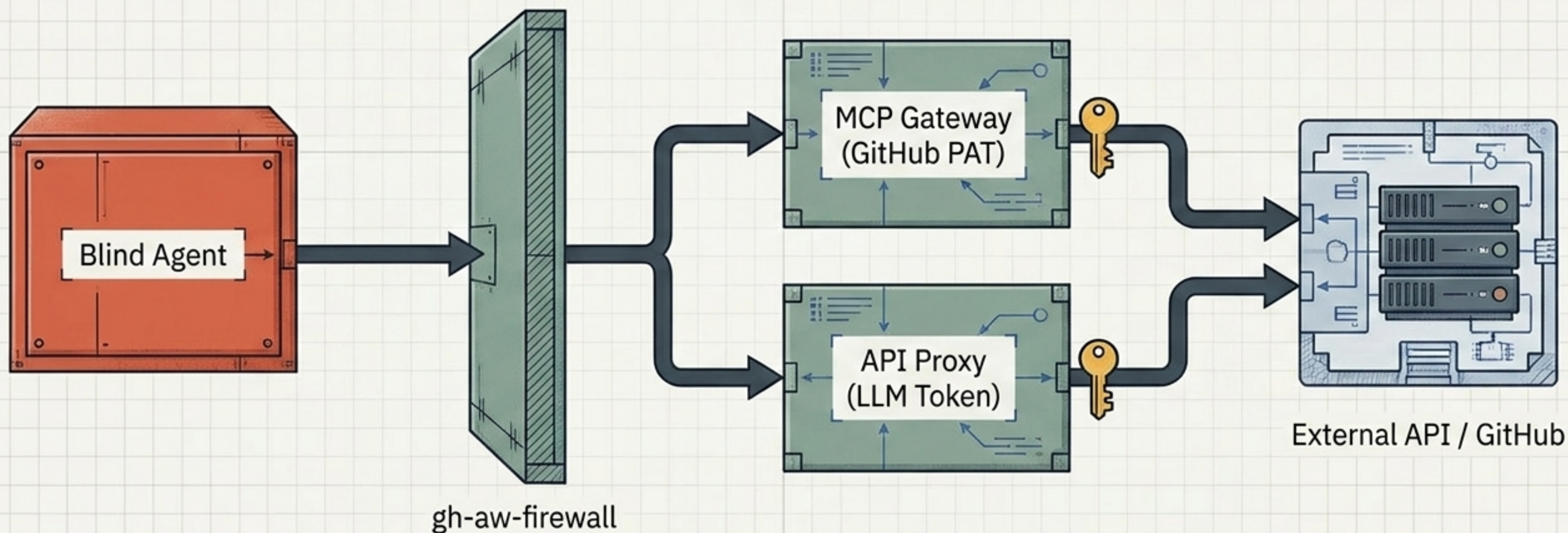
3 Layers of Distrust: เกราะป้องกันสามชั้น

ระบบถูกออกแบบให้แต่ละชั้นทำงานแยกจากกันอย่างอิสระ หากชั้นใดชั้นหนึ่งมีช่องโหว่ หรือ Agent หลุดจากการควบคุม เกราะชั้นต่อไปจะยังคงทำหน้าที่กักกันความเสียหายได้อย่างสมบูรณ์



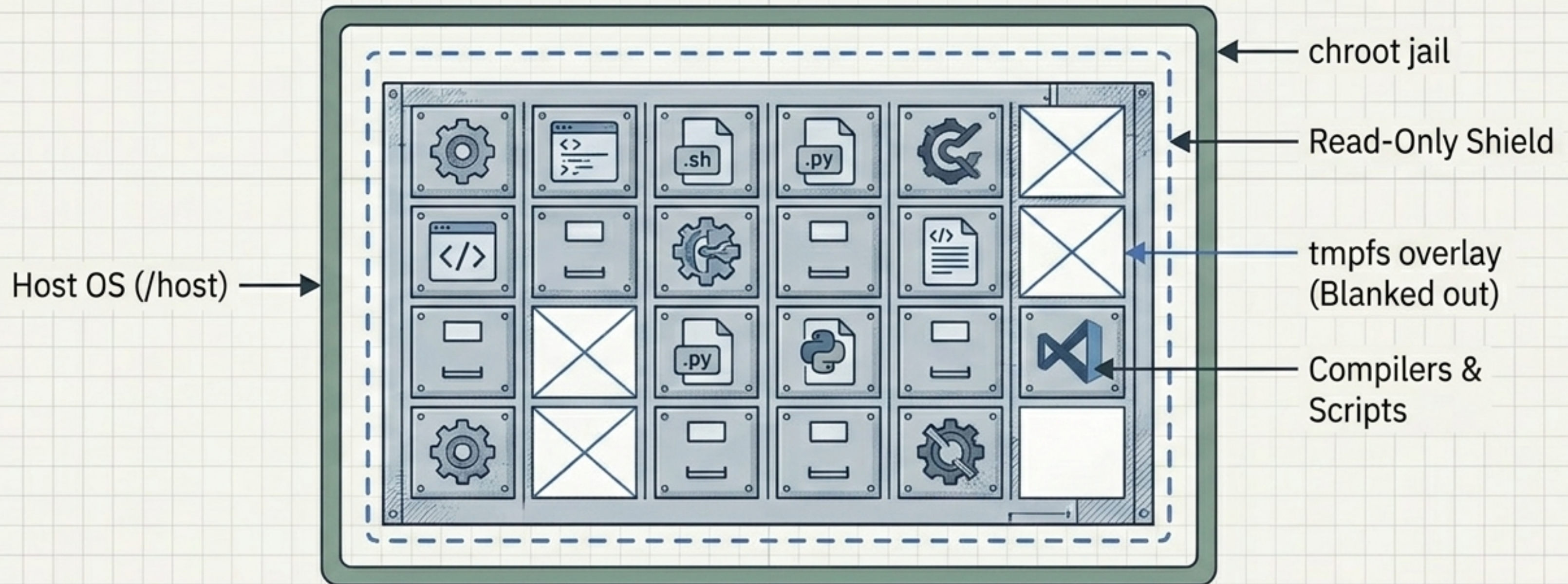
Zero-Secret Topology: ตัดขาดการเข้าถึงความลับ

Agent ไม่มีความจำเป็นต้องรู้รหัสผ่านใดๆ โทเคนทั้งหมดจะถูกเก็บไว้ที่ Proxy และ Gateway แยกต่างหาก (Physical Separation) หาก Agent โดนแฮ็ก จะไม่มี Secret ใดๆ ให้ขโมยได้จากภายใน Container ของมันเลย



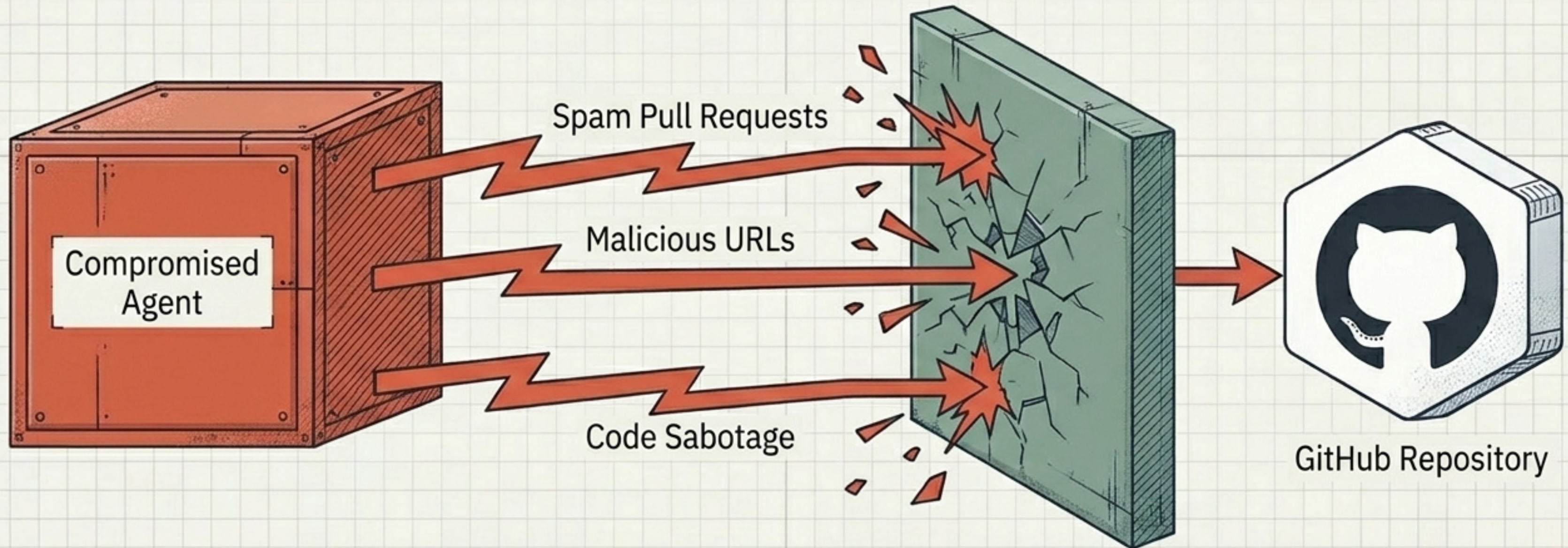
The Read-Only Jail: ให้ใช้เครื่องมือ แต่ปิดตาไม่ให้เห็นความลับ

Agent จำเป็นต้องใช้ Compiler และเครื่องมือบน Host VM เพื่อทำงาน แต่แทนที่จะก๊อปปี้ทุกอย่าง ระบบใช้วิธีแยก Host แบบ Read-only แล้วใช้ tmpfs บังคับโฟลเดอร์ที่เก็บความลับไว้ ทำให้ Agent มองเห็นแค่โฟลเดอร์ว่างเปล่า



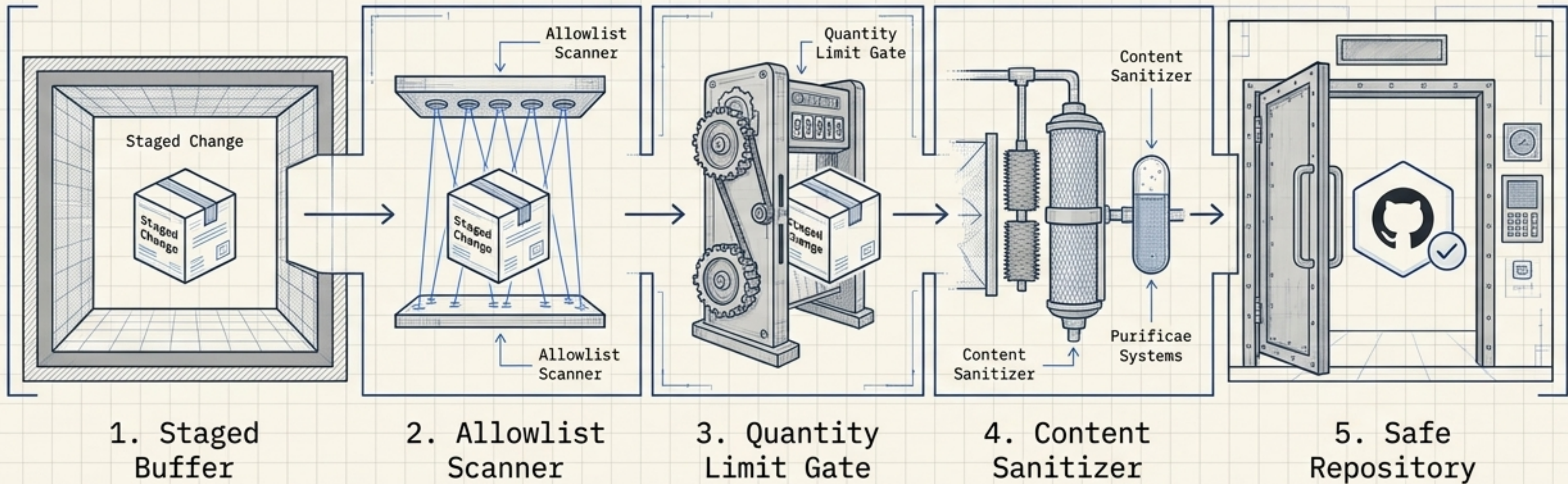
ความเสี่ยงภายนอก: เมื่อ Agent พยายามทำลายระบบจากภายนอก

การไม่มี Secret ไม่ได้แปลว่าไม่มีความเสี่ยง Agent ที่ถูกควบคุมด้วย Prompt Injection อาจพยายามส่งสแปม PR หรือฝังลิงก์อันตรายลงในโค้ด ทุกการกระทำภายนอก (Write Operations) จึงต้องถูกสกัดและตรวจสอบ



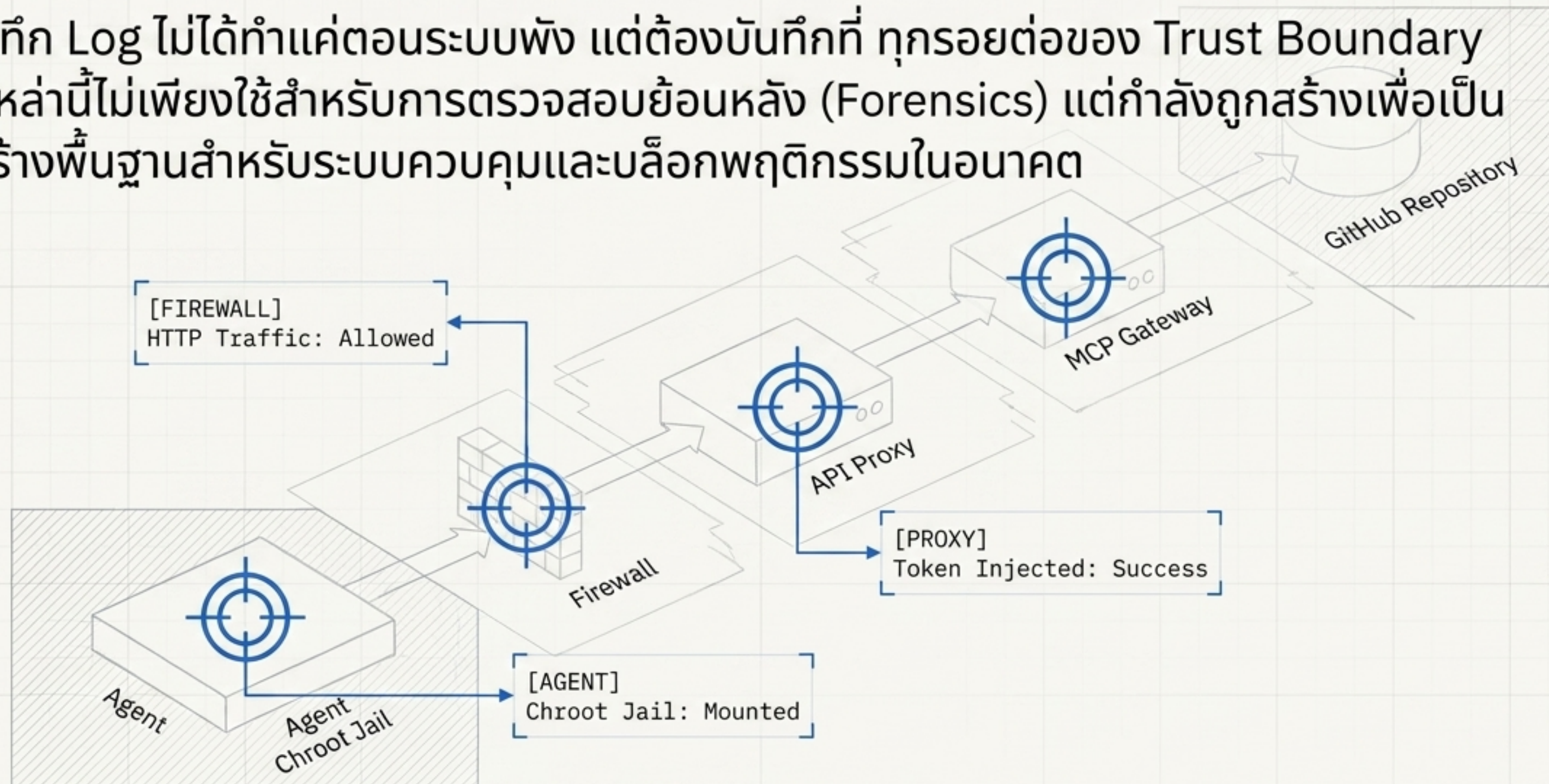
The Deterministic Airlock: ระบบกลั่นกรองผลลัพธ์

Agent ไม่สามารถเขียนข้อมูลลง GitHub ได้โดยตรง ทุกการเปลี่ยนแปลงจะถูกพักไว้ใน Buffer และต้องผ่านด่านวิเคราะห์ (Deterministic Analysis) 3 ขั้นตอนเสมอ หากไม่ผ่านเกณฑ์ สิ่งที่ Agent ทำจะถูกปิดกั้นทันที



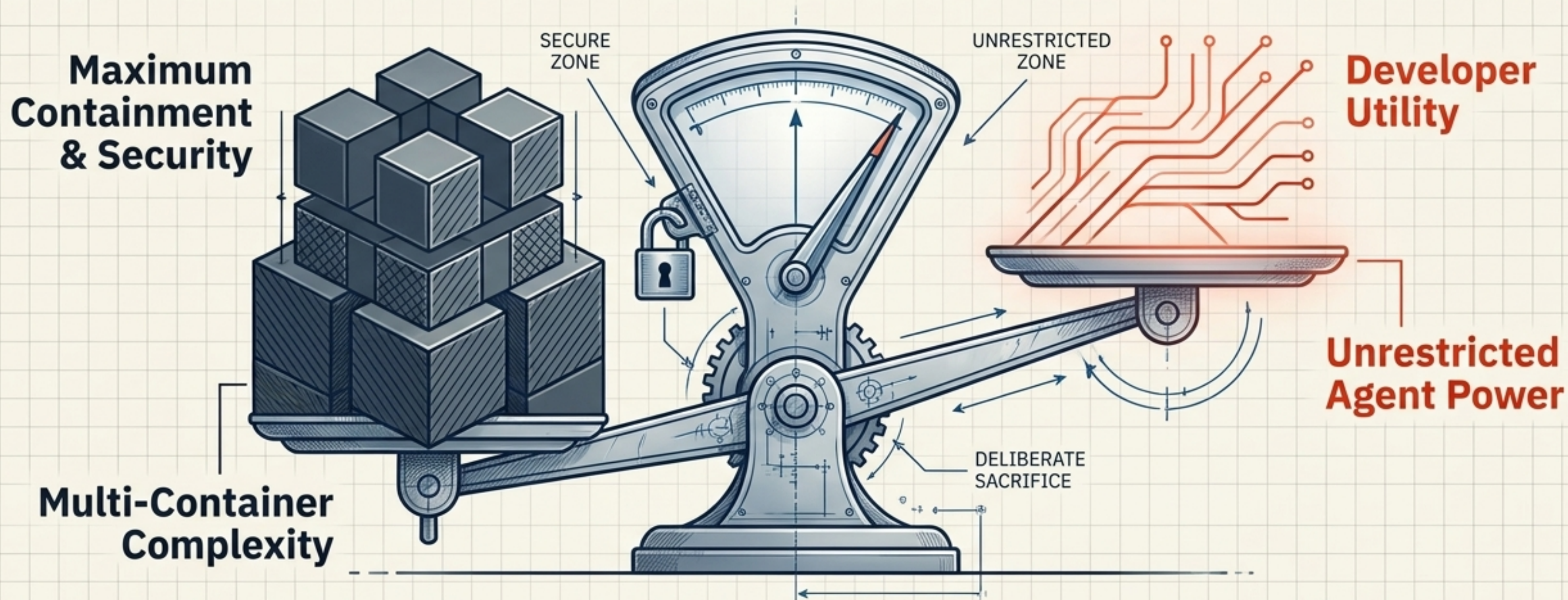
Observability เป็นหัวใจของ Control Plane

การบันทึก Log ไม่ได้ทำแค่ตอนระบบพัง แต่ต้องบันทึกที่ ทุกรอยต่อของ Trust Boundary ข้อมูลเหล่านี้ไม่เพียงใช้สำหรับการตรวจสอบย้อนหลัง (Forensics) แต่กำลังถูกสร้างเพื่อเป็นโครงสร้างพื้นฐานสำหรับระบบควบคุมและบล็อกพฤติกรรมในอนาคต

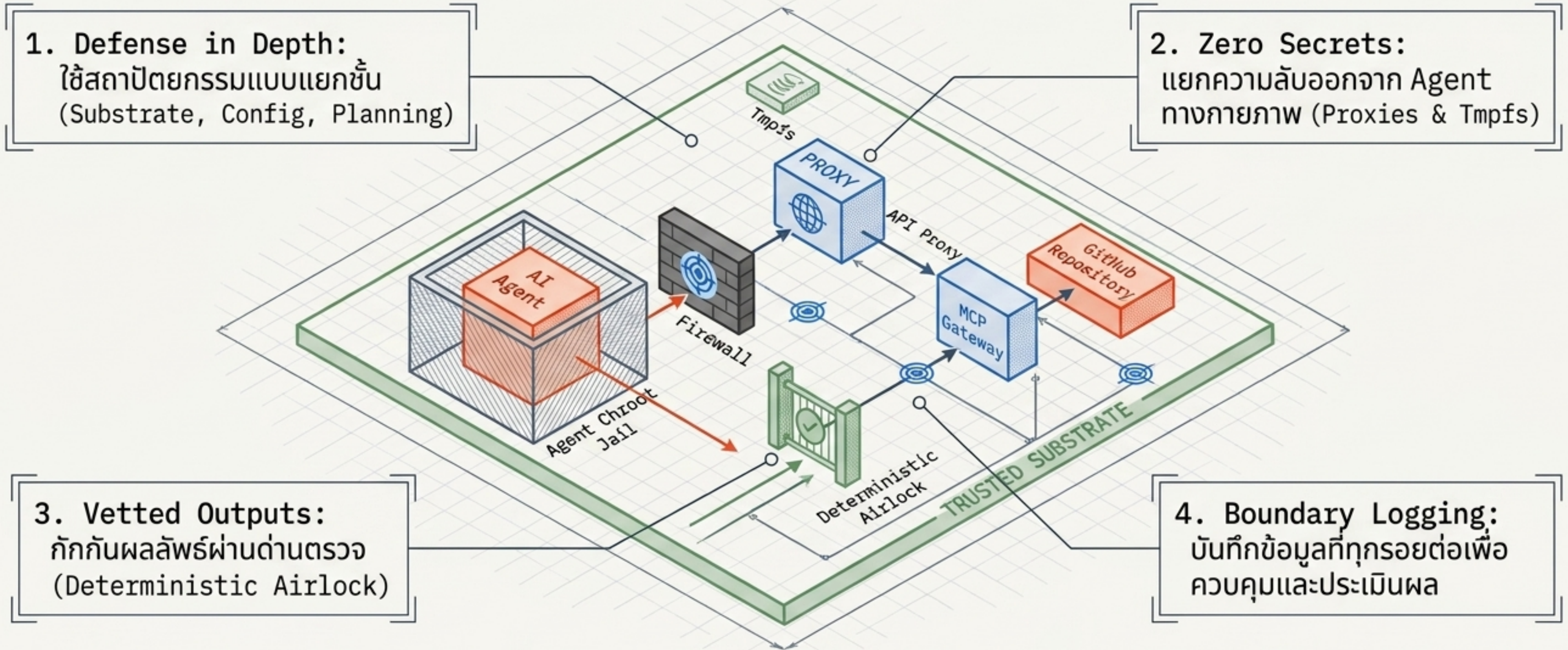


สิ่งที่ต้องแลก (The Trade-Off)

ไม่มีสถาปัตยกรรมใด ไร้ข้อจำกัด การบังคับใช้ Chroot, Firewall และระบบตรวจสอบ Airlock ทำให้ความสามารถบางอย่างของ Agent ถูกจำกัด สถาปัตยกรรมระดับนี้มีต้นทุนทางวิศวกรรมที่สูง (Engineering Overhead) ซึ่งเหมาะกับสเกลระดับองค์กรที่เน้นความปลอดภัยเป็นอันดับแรก



บทสรุป: 4 เสาหลักของความปลอดภัยแบบ Agentic



กรอบแนวคิดนี้ไม่ได้ใช้แค่กับ GitHub แต่คือพิมพ์เขียวสำหรับทุกองค์การที่จะนำ AI Agent มาใช้ใน Production